

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут»

Розрахунково-графічна робота
з дисципліни «Бази даних»

**«Створення додатку бази даних, орієнтованого на
взаємодію з СУБД PostgreSQL»**

Виконав студент групи: КВ-33

ПІБ: Ляшенко Д. Д.

Перевірив: Павловський В. І.

Київ 2025

Мета роботи

Метою даної роботи є здобуття вмінь програмування прикладних додатків баз даних PostgreSQL.

Постановка задачі

В рамках даної розрахунково-графічної роботи мають бути вирішені наступні задачі:

1. Реалізувати функції перегляду, внесення, редагування та вилучення даних у таблицях бази даних, створених у лабораторній роботі №1, засобами консольного інтерфейсу.
2. Передбачити автоматичне пакетне генерування «рандомізованих» даних у базі.
3. Забезпечити реалізацію пошуку за декількома атрибутами з двох та більше сутностей одночасно: для числових атрибутів – у рамках діапазону, для рядкових – як шаблон функції LIKE оператора SELECT SQL, для логічного типу – значення True/False, для дат – у рамках діапазону дат.
4. Програмний код виконати згідно шаблону MVC (модель-подання-контролер).

Опис предметної галузі

Тема бази даних: Онлайн-платформа для здачі та оренди нерухомості. На заданій платформі людина (користувач) може одночасно виступати як орендарем, так і тим, хто здає нерухомість в оренду. Користувач може як бронювати, так і здавати в оренду нерухомість. Нерухомість може мати різні відгуки від користувачів. У даної моделі платформи передбачені умови, за які нерухомість може бути орендованою.

При створенні даної бази даних було виділено такі сутності:

1. **Користувач (User)** – представляє клієнтів платформи:
 - Атрибути: ім'я, прізвище, електронна пошта, дата реєстрації.
2. **Нерухомість (Realty)** – представляє об'єкт нерухомості (квартира, будинок, тощо):
 - Атрибути: тип нерухомості (квартира, будинок, тощо), назва міста де знаходиться нерухомість, вулиця місцезнаходження нерухомості, статус нерухомості (здається в оренду, не здається в оренду, орендується), мінімальний строк оренди, депозит(0, сума депозиту), дозволені умови (примітка від власника нерухомості), ціна, термін внесення цінової політики (місяць, доба).
3. **Бронювання (Booking)** – представляє можливість орендування нерухомості:

– Атрибути: дата початку, дата завершення, статус (очікує підтвердження, підтверджено, відхилено, завершено), сума угоди.

4. **Відгуки (Review)** – представляє об'єкт оцінки нерухомості:

– Атрибути: рейтинг (1-5).

5. **Власник нерухомості (Property owner)** – представляє собою користувача, що здає нерухомість в оренду:

– Атрибути: ім'я, прізвище, електронна пошта, дата реєстрації.

Зв'язки:

Зв'язок «Власник» - «Нерухомість»:

- Тип зв'язку: 1 до N (один власник може мати безліч нерухомостей; одна нерухомість відноситься до одного власника).

Зв'язок «Нерухомість» - «Відгук»:

- Тип зв'язку: 1 до N (одна нерухомість може мати безліч відгуків; один відгук відноситься до однієї нерухомості).

Зв'язок «Користувач» - «Відгук»:

- Тип зв'язку: 1 до N (один користувач може залишати безліч відгуків; один відгук належить одному користувачу).

Зв'язок «Власник» - «Нерухомість» - «Користувач»:

- Даний зв'язок є сутністю з ім'ям Бронювання та відповідними йому атрибутами. Даний зв'язок є неділимим. Сенс цього зв'язку існує лише у повному наборі сутностей, які в ньому беруть участь. Розглянемо 3 зв'язки між 2 сутностями, які ми можемо отримати в рамках даного зв'язку:

Зв'язок «Власник» - «Бронювання»:

- Тип зв'язку: 1 до N (один власник може мати багато бронювань; одне бронювання належить конкретному власнику).

Зв'язок «Нерухомість» - «Бронювання»:

- Тип зв'язку: 1 до N (одна нерухомість може бути заброньованою багато разів; одне бронювання відноситься до однієї конкретної нерухомості).

Зв'язок «Користувач» - «Бронювання»:

- Тип зв'язку: 1 до N (один користувач може робити багато бронювань; одне бронювання відноситься до одного користувача).

Графічне подання концептуальної моделі «Сутність-зв'язок» (ER-діаграма, побудована за нотацією «Пташина лапка») зображено на рисунку:

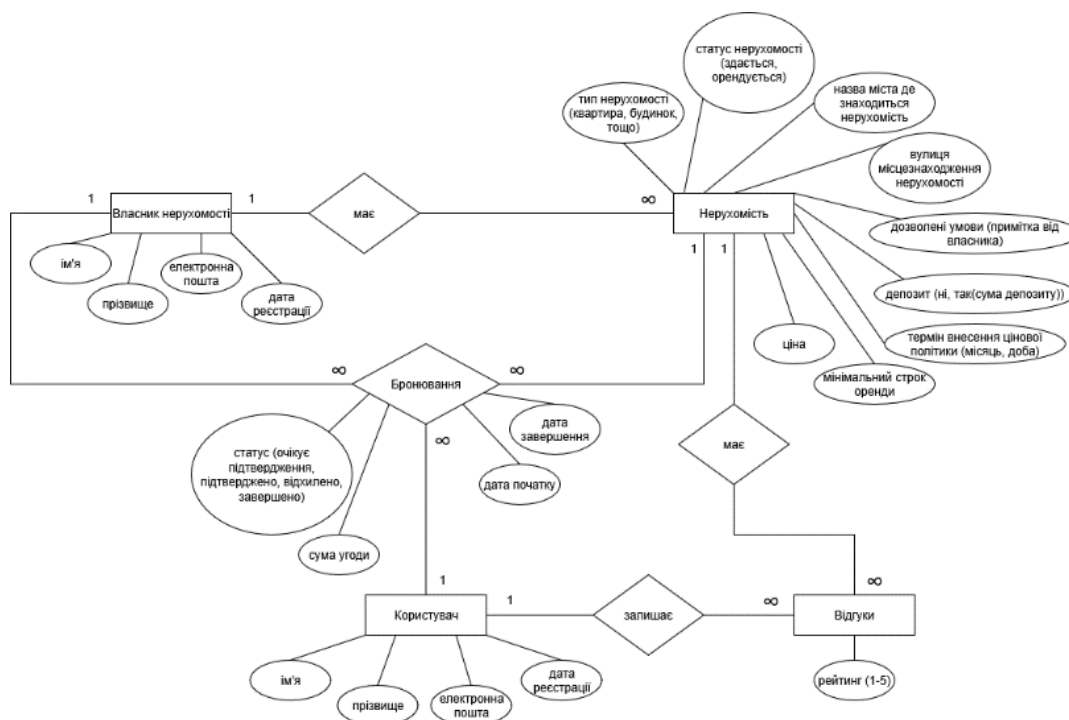


Рисунок 1 - Концептуальна модель «Сутність-зв'язок»

Перетворення концептуальної моделі у логічну модель та схему бази даних

Сутність "Користувач" перетворено в таблицю User з первинним ключем user_id та атрибутами first_name, last_name, data_registration, email.

Сутність "Нерухомість" перетворено в таблицю Realty з первинним ключем realty_id та атрибутами type_realty, status_realty, city_name, street_name, minimum_rental_period, deposit, permitted_conditions, price, payment_term.. У таблиці є зовнішній ключ property_owner_id, який пов'язаний з таблицею Property owner відповідно.

Сутність "Бронювання" перетворено в таблицю Booking з первинним ключем booking_id та атрибутами data_start, data_end, status_booking, price_booking. У таблиці є зовнішні ключі property_owner_id, user_id, realty_id, які пов'язані з таблицями Property owner, User, Realty відповідно.

Сутність "Відгуки" перетворено в таблицю Review з первинним ключем review_id та атрибутами rating. У таблиці є зовнішні ключі user_id та realty_id, які пов'язані з таблицями User та Realty відповідно.

Сутність "Власник нерухомості" перетворено в таблицю Property owner з первинним ключем property_owner_id та атрибутами first_name, last_name, data_registration, email.

Графічне подання логічної моделі «Сутність-зв'язок» зображено на рисунку:

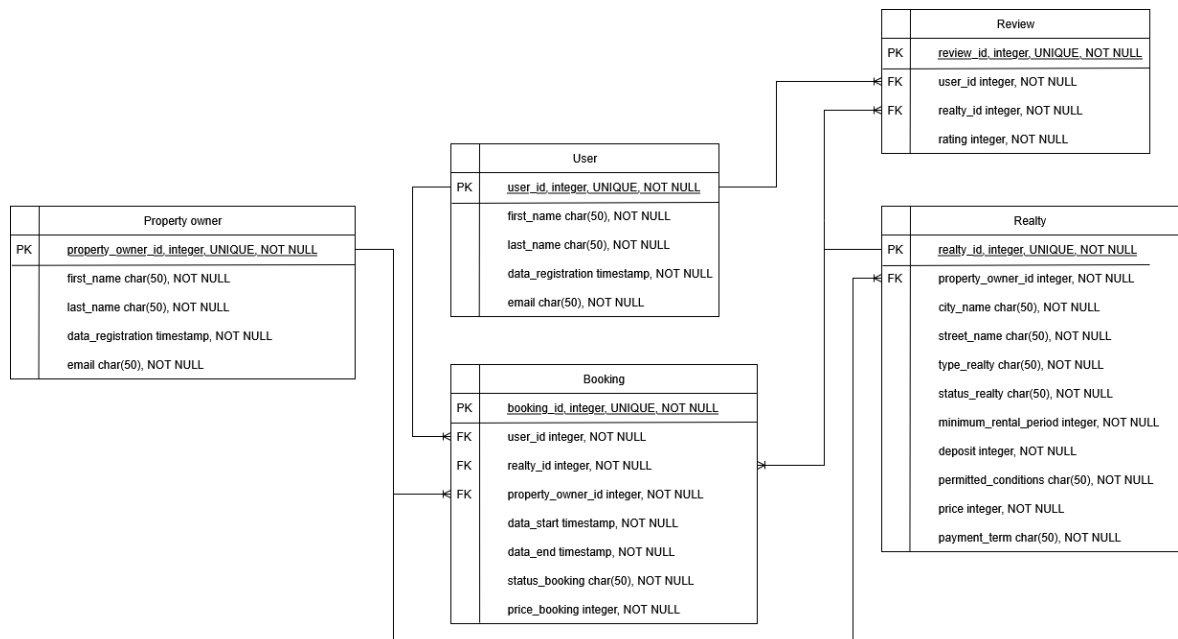


Рисунок 2 - Логічна модель «Сутність-зв'язок»

Графічне подання схеми бази даних у pgAdmin 4 зображено на рисунку:

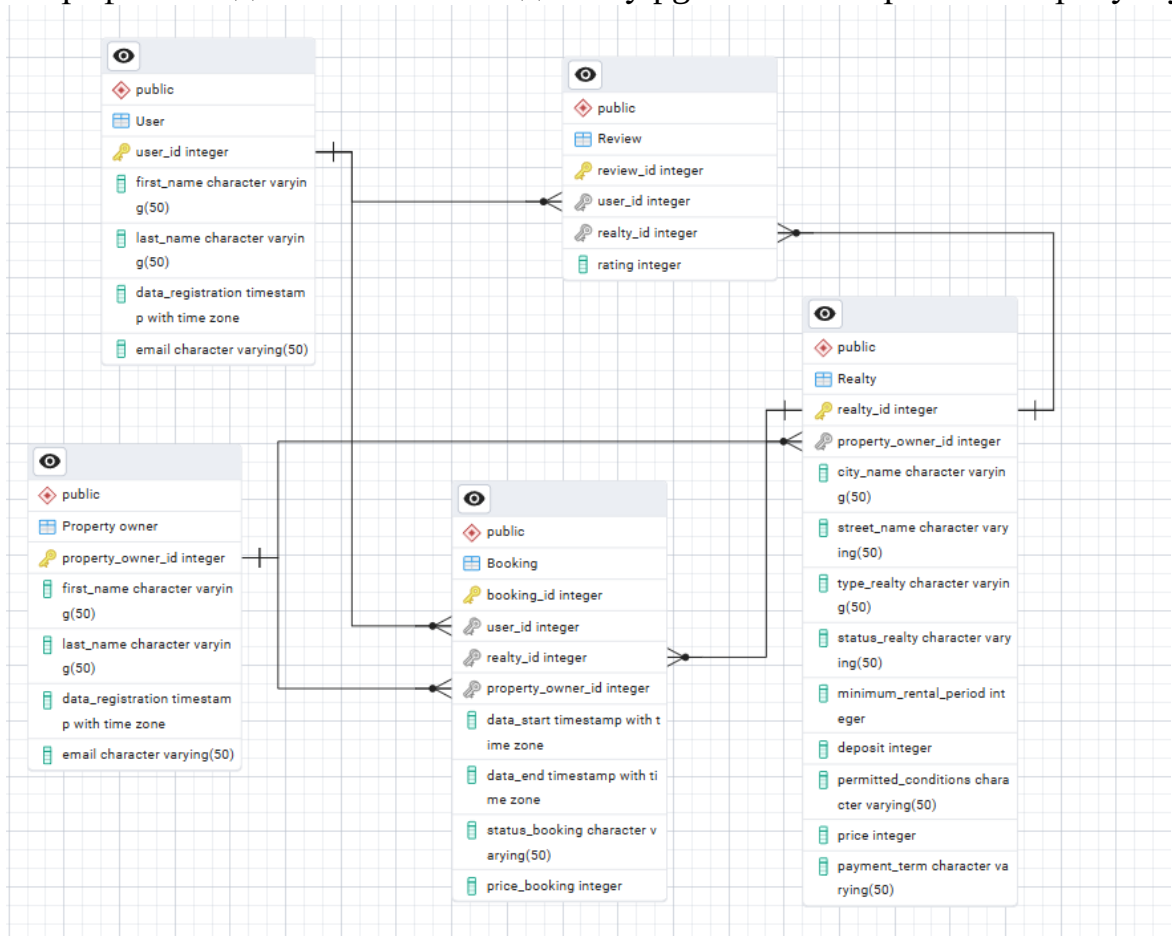


Рисунок 3 - Схема бази даних у pgAdmin 4

Середовище та компоненти розробки

Для розробки використовувалась мова програмування Python, середовище розробки Visual Studio Code, стороння бібліотека для доступу до PostgreSQL – psycopg2, вбудована бібліотека, задля можливості використання таймера спрацювання функцій – time, а також стороння бібліотека, що надає можливість коректного виводу інформації таблиць – tabulate. Встановити бібліотеку tabulate можна шляхом написання наступної команди до консолі користувача: `pip install --trusted-host pypi.org --trusted-host files.pythonhosted.org tabulate`

Шаблон проектування

MVC – Шаблон проектування, який був використаний при проектуванні застосунку.

Model – Представляє собою клас, що описує логіку використання даних, отриманих від користувача, а також тих даних, що початково закладені у саму програму.

View – Представляє собою клас комунікації із користувачем, а також виводу необхідної інформації.

Controller – Представляє собою клас, що описує головну логіку взаємодії усіх інших класів та модулів.

Структура програми та її опис

Подання структури програми зображено на рисунках нижче:

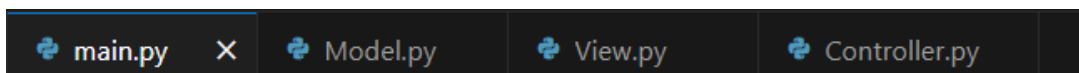


Рисунок 4 – Структура програми

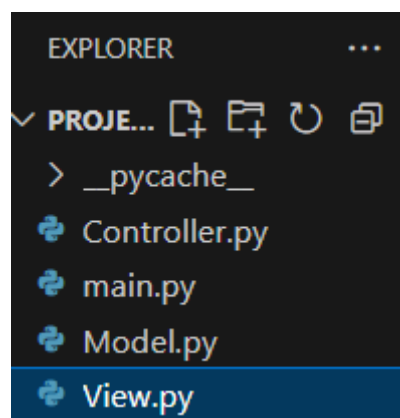


Рисунок 5 – Структура програми

Програма умовно поділена на 4 модулі:

1. Файл main.py;
2. Файл Model.py;
3. Файл View.py;
4. Файл Controller.py;

Файл main.py являє собою модуль початкової точки запуску усієї програми. Саме у цьому файлі відбувається підключення головного командного модуля MVC шаблону – Controller.py

Файл Controller.py являє собою модуль керування, що визначає порядок дій, а також порядок викликів необхідних функцій та методів інших модулів.

Файл View.py являє собою модуль комунікації із користувачем. Даний модуль виводить необхідну інформацію до користувача, отримує від користувача дані, після чого готує отримані дані до відправки в модуль Model.py.

Файл Model.py являє собою модуль взаємодії із базою даних. Даний модуль отримує дані від модуля View.py, після чого починає спілкування із базою даних, аналізує отримані дані, та надсилає їх до назад до модуля View.py, аби вивести результати до користувача.

Структура меню програми

Меню користувача складається із загальної інформації про структуру усіх наявних таблиць, їх стовпців та відповідних типів, а також із 5-ти пунктів, якими може скористатися користувач.

Консольне подання структури користувацького меню програми зображено на рисунку нижче:

```
### --- You can choose out of these tables:
### --- TABLE: Users
+-----+-----+-----+-----+-----+
| user_id | first_name | last_name | data_registration | email |
+-----+-----+-----+-----+-----+
| INT     | CHAR VAR  | CHAR VAR  | TS WITH TZ        | CHAR VAR |
+-----+-----+-----+-----+-----+
### --- TABLE: Review
+-----+-----+-----+-----+
| review_id | user_id | realty_id | rating |
+-----+-----+-----+-----+
| INT       | INT     | INT       | INT     |
+-----+-----+-----+-----+
### --- TABLE: Realty
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| realty_id | property_owner_id | city_name | street_name | type_realty | status_realty | minimum_rental_period | deposit | permitted_conditions | price | payment_term |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| INT       | INT               | CHAR VAR  | CHAR VAR    | CHAR VAR    | CHAR VAR      | INT                  | INT     | CHAR VAR             | INT  | CHAR VAR      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
### --- TABLE: Property owner
+-----+-----+-----+-----+-----+
| property_owner_id | first_name | last_name | data_registration | email |
+-----+-----+-----+-----+-----+
| INT               | CHAR VAR  | CHAR VAR  | TS WITH TZ        | CHAR VAR |
+-----+-----+-----+-----+-----+
### --- TABLE: Booking
+-----+-----+-----+-----+-----+-----+-----+
| booking_id | user_id | realty_id | property_owner_id | data_start | data_end | status_booking | price_booking |
+-----+-----+-----+-----+-----+-----+-----+
| INT        | INT     | INT       | INT               | TS WITH TZ | TS WITH TZ | CHAR VAR       | INT           |
+-----+-----+-----+-----+-----+-----+-----+

----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option:
```

Рисунок 6 – Структура меню програми

Нижче розглянемо код кожного модуля, проаналізуємо їх поведінку, можливі ситуації та результати роботи.

Модуль main.py

Даний модуль визначає початкову точку запуску програми

```
# Стартова точка роботи програми
from Controller import Controller

def main():
    controller = Controller()
    controller.run()

if __name__ == "__main__":
    main()
```

Модуль Controller.py

Даний модуль визначає порядок дій, порядок виклику необхідних методів та функцій у результаті внесених даних користувача та поведінки бази даних.

```
# Логіка спілкування з користувачем
from View import View
from Model import Model
from time import sleep

class Controller:
    def __init__(self):
        self.view = View()
        self.model = Model()
        self.main_input = ""
        self.action_input = ""

    def run(self):
        try:
            self.model.start_script_method()
        except:
            print("database is already created\n")
            sleep(3)
        while True:
            try:
                self.view.clear_console()
                self.view.show_base_info(self.model.get_tables_info())
                self.view.print_action_menu()
                self.action_input = self.view.get_input()
                match self.action_input:
                    case "1":
                        self.model.get_table_data(*self.view.get_table_description(
))
                        self.view.clear_console()
```



```

        self.view.show_table_data(*self.model.analysis_table_data()
)
        case "2":
            self.model.add_table_data(*self.view.get_ADD_description())
        case "3":
            self.model.delete_table_data(*self.view.get_DELETE_descript
ion())
        case "4":
            self.model.update_table_data(*self.view.get_UPDATE_descript
ion())
        case "5":
            self.model.generate_data(self.view.get_generate_info())
    except:
        print("\n# - Syntax ERROR . . .")
        sleep(2)
        self.model.connect.rollback()

```

Модуль View.py

Даний модуль виконує вивід необхідної інформації до користувача, а також отримує дані від самого користувача, після чого перенаправляє їх до модуля Model.py, задля наступного аналізу.

Ініціалізація роботи

```

def __init__(self):
    self.name_table = ""
    self.columns = ""
    self.conditions = ""
    self.order_by = ""
    self.number_rows = ""
    self.values = ""

```

Отримання обраної опції

```

def get_input(self):
    return input("Choose your option: ")

```

Очищення консолі

```

def clear_console(self):
    print("\033[2J\033[H", end="")

```

Аналіз отриманих даних від модуля Model.py

```

def get_types(self, number_table: int, info_list: list):
    return [(types[1] for types in info_list[number_table])]

def get_names(self, number_table: int, info_list: list):
    return [names[0] for names in info_list[number_table]]

```

Вивід меню користувача

```
def print_action_menu(self):
    print("\n----- MENU -----")
    print("1. Get table data")
    print("2. Add new data")
    print("3. Delete data")
    print("4. Update data")
    print("5. Generate random data")
```

Отримання опису таблиці задля виводу її витримки

```
def get_table_description(self):
    self.name_table = input("\nFROM 'table_name': ").strip()
    self.columns = input("SELECT 'columns_names' or 'ALL': ").strip()
    self.conditions = input("WHERE 'conditions' or 'NONE': ").strip()
    self.order_by = input("ORDER BY 'columns_names' or 'NONE': ").strip()
    self.number_rows = input("LIMIT 'number_rows' or 'ALL': ").strip()
    return self.name_table, self.columns, self.number_rows, self.order_by,
self.conditions
```

Отримання кількості згенерованих даних

```
def get_generate_info(self):
    return input("Enter number of new data: ").strip()
```

Отримання опису додавання, видалення, оновлення даних відповідно

```
def get_ADD_description(self):
    self.name_table = input("\nINSERT INTO 'table_name': ").strip()
    self.values = input("VALUES 'value_1,value_2,...;': ").strip()
    return self.name_table, self.values

def get_DELETE_description(self):
    self.name_table = input("FROM 'table_name': ").strip()
    self.conditions = input("WHERE 'conditions': ").strip()
    return self.name_table, self.conditions

def get_UPDATE_description(self):
    self.name_table = input("UPDATE 'table_name': ").strip()
    self.columns = input("SET 'column_1, column_2, column_3, ...': ").strip()
    self.values = input("VALUES 'value_1, value_2, value_3, ...': ").strip()
    self.conditions = input("WHERE 'conditions': ").strip()
    return self.name_table, self.columns, self.values, self.conditions
```

Вивід загальної інформації про структуру таблиць та типів до відповідних колонок

```
def show_base_info(self, info_list: list):
    print("### --- You can choose out of these tables:")
```

```

        print("### --- TABLE: Users")
        print(tabulate(self.get_types(0, info_list), headers = self.get_names(0,
info_list), tablefmt = "psql"))
        print("### --- TABLE: Review")
        print(tabulate(self.get_types(1, info_list), headers = self.get_names(1,
info_list), tablefmt = "psql"))
        print("### --- TABLE: Realty")
        print(tabulate(self.get_types(2, info_list), headers = self.get_names(2,
info_list), tablefmt = "psql"))
        print("### --- TABLE: Property owner")
        print(tabulate(self.get_types(3, info_list), headers = self.get_names(3,
info_list), tablefmt = "psql"))
        print("### --- TABLE: Booking")
        print(tabulate(self.get_types(4, info_list), headers = self.get_names(4,
info_list), tablefmt = "psql"))

```

Вивід конкретних даних таблиці за бажанням користувача

```

def show_table_data(self, columns_names: list, data_row: list):
    print(f'### --- TABLE: {self.name_table}')
    print(tabulate(data_row, headers = columns_names, tablefmt = "psql"))
    input()

```

Модуль Model.py

Даний модуль відповідає за головний аналіз отриманих даних від користувача, комунікацію із базою даних, а також надсилення результатів аналізу та спілкування із базою даних до модуля виводу інформації.

Ініціалізація

```

def __init__(self):
    self.connect = psycopg.connect(
        dbname="postgres",
        user="postgres",
        password="11111",
        host="localhost",
        port="5432"
    )
    self.cursor = self.connect.cursor()
    self.types_dict = {
        23: "INT",          # integer
        1043: "CHAR VAR",   # character varying
        1184: "TS WITH TZ"  # timestamp with timezone
    }
    self.search_fk_query = """
SELECT
    kcu.column_name AS foreign_key_column,
    ccu.table_name AS parent_table,
    ccu.column_name AS parent_column
FROM

```

```

        information_schema.key_column_usage AS kcu
JOIN
        information_schema.referential_constraints AS rc
        ON kcu.constraint_name = rc.constraint_name
JOIN
        information_schema.constraint_column_usage AS ccu
        ON rc.unique_constraint_name = ccu.constraint_name
WHERE
        kcu.table_schema = 'public'
        AND kcu.table_name = %s;
"""

self.start_script = """
BEGIN;

CREATE TABLE IF NOT EXISTS public."Property owner"
(
property_owner_id integer NOT NULL,
first_name character varying(50) NOT NULL,
last_name character varying(50) NOT NULL,
data_registration timestamp with time zone NOT NULL,
email character varying(50) NOT NULL,
CONSTRAINT property_owner_id_pk PRIMARY KEY (property_owner_id)
);
CREATE TABLE IF NOT EXISTS public."Users"
(
user_id integer NOT NULL,
first_name character varying(50) NOT NULL,
last_name character varying(50) NOT NULL,
data_registration timestamp with time zone NOT NULL,
email character varying(50) NOT NULL,
CONSTRAINT user_id_pk PRIMARY KEY (user_id)
);
CREATE TABLE IF NOT EXISTS public."Realty"
(
realty_id integer NOT NULL,
property_owner_id integer NOT NULL,
city_name character varying(50) NOT NULL,
street_name character varying(50) NOT NULL,
type_realty character varying(50) NOT NULL,
status_realty character varying(50) NOT NULL,
minimum_rental_period integer NOT NULL,
deposit integer NOT NULL,
permitted_conditions character varying(50) NOT NULL,
price integer NOT NULL,
payment_term character varying(50) NOT NULL,
CONSTRAINT realty_id_pk PRIMARY KEY (realty_id)
);
CREATE TABLE IF NOT EXISTS public."Review"
(
review_id integer NOT NULL,
user_id integer NOT NULL,
realty_id integer NOT NULL,
rating integer NOT NULL,

```

```
CONSTRAINT review_id_pk PRIMARY KEY (review_id)
);
CREATE TABLE IF NOT EXISTS public."Booking"
(
    booking_id integer NOT NULL,
    user_id integer NOT NULL,
    realty_id integer NOT NULL,
    property_owner_id integer NOT NULL,
    data_start timestamp with time zone NOT NULL,
    data_end timestamp with time zone NOT NULL,
    status_booking character varying(50) NOT NULL,
    price_booking integer NOT NULL,
    CONSTRAINT booking_id_pk PRIMARY KEY (booking_id)
);
ALTER TABLE IF EXISTS public."Realty"
ADD CONSTRAINT property_owner_fk FOREIGN KEY (property_owner_id)
REFERENCES public."Property owner" (property_owner_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;

ALTER TABLE IF EXISTS public."Review"
ADD CONSTRAINT user_id_fk FOREIGN KEY (user_id)
REFERENCES public."Users" (user_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;

ALTER TABLE IF EXISTS public."Review"
ADD CONSTRAINT realty_id_fk FOREIGN KEY (realty_id)
REFERENCES public."Realty" (realty_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;

ALTER TABLE IF EXISTS public."Booking"
ADD CONSTRAINT user_id_fk FOREIGN KEY (user_id)
REFERENCES public."Users" (user_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;

ALTER TABLE IF EXISTS public."Booking"
ADD CONSTRAINT realty_id_fk FOREIGN KEY (realty_id)
REFERENCES public."Realty" (realty_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;

ALTER TABLE IF EXISTS public."Booking"
ADD CONSTRAINT property_owner_id_fk FOREIGN KEY (property_owner_id)
REFERENCES public."Property owner" (property_owner_id) MATCH SIMPLE
```

```

ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;
END;
"""

```

Запуск створення таблиць, за умови їх відсутності

```

def start_script_method(self):
    self.cursor.execute(self.start_script)
    self.connect.commit()

```

Метод генерування запиту отримання інформації про структуру таблиці

```

def get_table_schema_info(self, table_name: str) -> list:
    command = f'SELECT * FROM "{table_name}" LIMIT 0;'
    self.cursor.execute(command)
    columns_names = [column_info[0] for column_info in self.cursor.description]
    types_row = [self.types_dict.get(column_info[1], "UNKNOWN") for column_info
in self.cursor.description]
    return list(zip(columns_names, types_row))

```

Запуск попереднього методу

```

def get_tables_info(self) -> list:
    table_names = ["Users", "Review", "Realty", "Property owner", "Booking"]
    return [self.get_table_schema_info(name) for name in table_names]

```

Модуль генерування даних

```

def generate_data(self, number_of_rows_str: str):
    try:
        num = int(number_of_rows_str)
        if num <= 0:
            print("\n# -> Кількість рядків повинна бути додатним числом.")
            input("Натисніть Enter для продовження...")
            return
    except (ValueError, TypeError):
        print("\n# -> ПОМИЛКА: Введіть коректне число.")
        input("Натисніть Enter для продовження...")
        return
    users_query = """
INSERT INTO "Users" (user_id, first_name, last_name, data_registration,
email)
SELECT
    row_number() OVER () + (SELECT COALESCE(MAX(user_id), 0) FROM "Users"),
    fname,
    lname,
    NOW() - (random() * interval '365 days'),

```

```

        lower(fname || '.' || lname || (random()*1000)::int) ||
'@generated.com'
    FROM
        (SELECT unnest(array['John', 'Jane', 'Peter', 'Alice', 'Chris', 'Eva'])
as fname) f,
        (SELECT unnest(array['Smith', 'Doe', 'Jones', 'Williams', 'Brown',
'Davis']) as lname) l
    ORDER BY random() LIMIT %s;
"""

property_owner_query = """
INSERT INTO "Property owner" (property_owner_id, first_name, last_name,
data_registration, email)
SELECT
    row_number() OVER () + (SELECT COALESCE(MAX(property_owner_id), 0) FROM
"Property owner"),
    fname,
    lname,
    NOW() - (random() * interval '365 days'),
    lower(fname || '.' || lname || (random()*1000)::int) || '@owner.com'
FROM
    (SELECT unnest(array['Michael', 'Sarah', 'David', 'Laura', 'James',
'Linda']) as fname) f,
    (SELECT unnest(array['Miller', 'Wilson', 'Moore', 'Taylor', 'Anderson',
'Thomas']) as lname) l
    ORDER BY random() LIMIT %s;
"""

realty_query = """
INSERT INTO "Realty" (realty_id, property_owner_id, city_name, street_name,
type_realty, status_realty,
                        minimum_rental_period, deposit, permitted_conditions,
price, payment_term)
SELECT
    n + (SELECT COALESCE(MAX(realty_id), 0) FROM "Realty"),
    (SELECT property_owner_id FROM "Property owner" ORDER BY random() LIMIT
1),
    (SELECT city FROM unnest(array['Kyiv', 'Lviv', 'Odesa', 'Kharkiv']))
city ORDER BY random() LIMIT 1),
    (SELECT street FROM unnest(array['Main St', 'Oak Ave', 'Pine Ln',
'Maple Dr'])) street ORDER BY random() LIMIT 1),
    (SELECT type FROM unnest(array['Apartment', 'House', 'Cottage'])) type
ORDER BY random() LIMIT 1),
    (SELECT status FROM unnest(array['Available', 'Rented', 'Under
repair'])) status ORDER BY random() LIMIT 1),
    (1 + random() * 11)::int,
    (1000 + random() * 4000)::int,
    (SELECT cond FROM unnest(array['Pets allowed', 'No smoking', 'Families
only'])) cond ORDER BY random() LIMIT 1),
    (5000 + random() * 15000)::int,
    (SELECT term FROM unnest(array['Monthly', 'Quarterly'])) term ORDER BY
random() LIMIT 1)
FROM generate_series(1, %s) as s(n);

```

```

"""

review_query = """
INSERT INTO "Review" (review_id, user_id, realty_id, rating)
SELECT
    n + (SELECT COALESCE(MAX(review_id), 0) FROM "Review"),
    (SELECT user_id FROM "Users" ORDER BY random() LIMIT 1),
    (SELECT realty_id FROM "Realty" ORDER BY random() LIMIT 1),
    (1 + random() * 4)::int
FROM generate_series(1, %s) as s(n);
"""

booking_query = """
INSERT INTO "Booking" (booking_id, user_id, realty_id, property_owner_id,
data_start, data_end,
                        status_booking, price_booking)
SELECT
    row_number() OVER () + (SELECT COALESCE(MAX(booking_id), 0) FROM
"Booking"),
    (SELECT user_id FROM "Users" ORDER BY random() LIMIT 1),
    realty.realty_id,
    realty.property_owner_id,
    NOW() + (random() * interval '10 days'),
    NOW() + (random() * interval '20 days' + interval '11 days'),
    (SELECT status FROM unnest(array['Confirmed', 'Pending', 'Cancelled'])
status ORDER BY random() LIMIT 1),
    (5000 + random() * 15000)::int
FROM "Realty" as realty
ORDER BY random() LIMIT %s;
"""

queries_in_order = [
    ("Users", users_query),
    ("Property owner", property_owner_query),
    ("Realty", realty_query),
    ("Review", review_query),
    ("Booking", booking_query)
]

try:
    print("\n# -> Починаю генерацію даних...")
    for table_name, query in queries_in_order:
        if table_name in ["Realty", "Review", "Booking"]:
            parent_check = {
                "Realty": ('SELECT 1 FROM "Property owner" LIMIT 1',
"Property owner"),
                "Review": ('SELECT 1 FROM "Users" LIMIT 1', "Users"),
                "Booking": ('SELECT 1 FROM "Realty" LIMIT 1', "Realty")
            }
            if table_name in parent_check:
                check_q, parent_name = parent_check[table_name]
                self.cursor.execute(check_q)
                if self.cursor.fetchone() is None:

```



```

        print(f"# -> Пропускаю \"{table_name}\", оскільки
        батьківська таблиця \"{parent_name}\" порожня.")
        continue

        print(f"# -> Заповнюю таблицю \"{table_name}\"...")
        self.cursor.execute(query, (num,))

        self.connect.commit()
        print("\n# -> Успішно згенеровано та додано дані до всіх таблиць!")

    except psycopg.Error as e:
        print(f"\n# -> СТАЛАСЯ ПОМИЛКА: {e.diag.message_primary}")
        print("# -> Відкочую всі зміни, щоб зберегти цілісність бази даних.")
        self.connect.rollback()

    input("Натисніть Enter для продовження...")

```

Метод формування та перевірки коректного занесення даних до таблиці

```

def _insert_data_safely(self, table_name: str, data: list):
    schema_info = self.get_table_schema_info(table_name)
    columns = [col[0] for col in schema_info]
    cols_str = ", ".join([f'"{c}"' for c in columns])
    placeholders = ", ".join(["%s"] * len(columns))
    insert_query = f'INSERT INTO "{table_name}" ({cols_str}) VALUES
    ({placeholders})'
    try:
        self.cursor.executemany(insert_query, data)
        self.connect.commit()
        print("\n# -> Дані успішно додано!")
    except Exception as e:
        print(f"\n# -> Помилка під час вставки: {e}")
        self.connect.rollback()
    input("Натисніть Enter для продовження...")

```

Метод занесення даних до таблиці

```

def add_table_data(self, name_table: str, values: str):
    schema_info = self.get_table_schema_info(name_table)
    columns_names = [col[0] for col in schema_info]
    # Конвертуємо вхідний рядок у список кортежів
    try:
        data_to_insert = [tuple(row.split(',')) for row in
        values.strip(';').split(';')]
    except ValueError:
        print("\n# -> Помилка формату вхідних даних.")
        input("Натисніть Enter для продовження...")
        return

    # Знаходимо всі зовнішні ключі для цієї таблиці
    self.cursor.execute(self.search_fk_query, (name_table,))
    foreign_keys = self.cursor.fetchall()
    if not foreign_keys:

```

```

        # Якщо зовнішніх ключів немає, просто вставляємо дані
        self._insert_data_safely(name_table, data_to_insert)
        return

    valid_rows_to_insert = []
    for row_data in data_to_insert:
        is_row_valid = True
        for fk in foreign_keys:
            fk_column_name, parent_table, parent_column = fk
            try:
                # Знаходимо індекс стовпця, що є зовнішнім ключем
                fk_index = columns_names.index(fk_column_name)
                value_to_check = row_data[fk_index]

                # Безпечний запит для перевірки існування батьківського запису
                check_query = f'SELECT 1 FROM "{parent_table}" WHERE
"{parent_column}" = %s LIMIT 1;'
                self.cursor.execute(check_query, (value_to_check,))

                if self.cursor.fetchone() is None:
                    print(f'\n# -> ПОМИЛКА: Значення "{value_to_check}" для
стовпця "{fk_column_name}"')
                    print(f'# -> не існує в батьківській таблиці
"{parent_table}". Рядок не буде додано.')
                    is_row_valid = False
                    break # Перериваємо перевірку для поточного рядка
            except ValueError:
                print(f"# -> КРИТИЧНА ПОМИЛКА: Стовпець {fk_column_name} не
знайдено в таблиці {name_table}")
                is_row_valid = False
                break

        if is_row_valid:
            valid_rows_to_insert.append(row_data)

    if valid_rows_to_insert:
        self._insert_data_safely(name_table, valid_rows_to_insert)
    else:
        print("\n# -> Жодного рядка не було додано через помилки валідації.")
        input("Натисніть Enter для продовження...")

```

Метод видалення даних з таблиці

```

def delete_table_data(self, name_table: str, conditions: str):
    # Перевіряємо, чи умови не порожні
    if not conditions.strip():
        print("\n# -> ПОМИЛКА: Умови для видалення не можуть бути порожніми!")
        input("Натисніть Enter для продовження...")
        return

    # Створюємо базову команду DELETE
    command = f'DELETE FROM "{name_table}" WHERE {conditions}'
    try:
        # Особлива логіка ТІЛЬКИ для батьківської таблиці "Realty"

```

```

        if name_table == "Realty":
            # 1. Знайти ID об'єктів нерухомості, які підпадають під умови
            # видалення
            select_ids_query = f'SELECT "realty_id" FROM "Realty" WHERE
            {conditions}'

            self.cursor.execute(select_ids_query)
            realty_ids_to_delete = self.cursor.fetchall()
            if not realty_ids_to_delete:
                print("\n# -> Жодного запису не знайдено за вашими умовами.")
                input("Натисніть Enter для продовження...")
                return

            # Перетворюємо список кортежів [(1,), (2,)] у простий список [1, 2]
            ids_list = [item[0] for item in realty_ids_to_delete]
            # 2. Перевірити, чи існують посилання на ці ID в дочірніх таблицях
            # Перевірка в таблиці "Review"
            check_review_query = 'SELECT 1 FROM "Review" WHERE "realty_id" =
            ANY(%s) LIMIT 1'
            self.cursor.execute(check_review_query, (ids_list,))
            if self.cursor.fetchone():
                print(f'\n# -> ПОМИЛКА: Неможливо видалити. Існують відгуки,
                пов'язані з цією нерухомістю.')
                self.connect.rollback()
                input("Натисніть Enter для продовження...")
                return

            # Перевірка в таблиці "Booking"
            check_booking_query = 'SELECT 1 FROM "Booking" WHERE "realty_id" =
            ANY(%s) LIMIT 1'
            self.cursor.execute(check_booking_query, (ids_list,))
            if self.cursor.fetchone():
                print(f'\n# -> ПОМИЛКА: Неможливо видалити. Існують бронювання,
                пов'язані з цією нерухомістю.')
                self.connect.rollback()
                input("Натисніть Enter для продовження...")
                return

            # 3. Виконуємо видалення (для "Realty" - після перевірок, для інших -
            # одразу)
            self.cursor.execute(command)
            # Перевіряємо, чи було щось видалено
            if self.cursor.rowcount > 0:
                print(f"\n# -> Успішно видалено {self.cursor.rowcount} рядків.")
                self.connect.commit() # Підтверджуємо транзакцію
            else:
                print("\n# -> Жодного рядка не було видалено (можливо, умови не
                знайшли збігів).")
                self.connect.rollback()
            except psycopg.Error as e:
                print(f"\n# -> Сталася помилка бази даних: {e}")
                self.connect.rollback() # Відкочуємо зміни у разі будь-якої помилки
                input("Натисніть Enter для продовження...")

```

Метод оновлення даних в таблиці

```

def update_table_data(self, name_table: str, columns: str, values: str, conditions:
str):
    # 1. Валідація та парсинг вхідних даних
    if not all([name_table, columns, values, conditions]):
        print("\n# -> ПОМИЛКА: Всі поля (таблиця, стовпці, значення, умови)
мають бути заповнені!")
        input("Натисніть Enter для продовження...")
        return

    cols_list = [c.strip() for c in columns.split(',')]
    vals_list = [v.strip() for v in values.split(',')]

    if len(cols_list) != len(vals_list):
        print("\n# -> ПОМИЛКА: Кількість стовпців не відповідає кількості
значень!")
        input("Натисніть Enter для продовження...")
        return
    try:
        # 2. Перевірка цілісності даних ПЕРЕД виконанням запиту
        self.cursor.execute(self.search_fk_query, (name_table,))
        foreign_keys = {fk[0]: (fk[1], fk[2]) for fk in self.cursor.fetchall()}
        for i, col_name in enumerate(cols_list):
            if col_name in foreign_keys:
                parent_table, parent_column = foreign_keys[col_name]
                value_to_check = vals_list[i]
                # Перевіряємо, чи існує нове значення в батьківській таблиці
                check_query = f'SELECT 1 FROM "{parent_table}" WHERE
"{parent_column}" = %s LIMIT 1;'
                self.cursor.execute(check_query, (value_to_check,))
                if self.cursor.fetchone() is None:
                    print(f'\n# -> ПОМИЛКА: Неможливо оновити стовпець
"{col_name}"')
                    print(f'# -> Значення "{value_to_check}" не існує в
батьківській таблиці "{parent_table}"')
                    self.connect.rollback()
                    input("Натисніть Enter для продовження...")
                    return
                set_clause = ", ".join([f'"{col}" = %s' for col in cols_list])
                command = f'UPDATE "{name_table}" SET {set_clause} WHERE {conditions}'
                # Передаємо значення для SET як параметри, щоб уникнути ін'єкцій
                self.cursor.execute(command, tuple(vals_list))
                if self.cursor.rowcount > 0:
                    print(f"\n# -> Успішно оновлено {self.cursor.rowcount} рядків.")
                    self.connect.commit()
                else:
                    print("\n# -> Жодного рядка не було оновлено (умови не знайшли
збігів).")
                    self.connect.rollback()
        except psycopg.Error as e:
            self.connect.rollback()
            # Надаємо користувачу зрозуміле повідомлення про помилку
            if e.pgcode == '23503': # Код помилки для foreign_key_violation
                print(f"\n# -> ПОМИЛКА ОНОВЛЕННЯ: Неможливо змінити цей запис,")

```

```

        print(f"# -> оскільки на нього посилаються дані в інших таблицях
(наприклад, бронювання або відгуки).")
    else:
        print(f"\n# -> Сталася помилка бази даних: {e}")
    input("Натисніть Enter для продовження...")

```

Метод генерування запиту отримання конкретних даних з таблиці

```

def get_table_data(self, name_table: str, columns: str, number_rows: str, order:
str, conditions: str):
    selected_columns = columns if columns != "ALL" else "*"
    command = f'SELECT {selected_columns} FROM "{name_table}"'
    if conditions != "NONE":
        command += f' WHERE {conditions}'
    if order != "NONE":
        command += f' ORDER BY {order}'
    if number_rows != "ALL":
        if number_rows.isdigit():
            command += f' LIMIT {number_rows}'

    self.cursor.execute(command)

```

Метод аналізу та надсилання отриманих даних до модуля виводу View.py

```

def analysis_table_data(self) -> tuple:
    if not self.cursor.description:
        return [], []
    columns_names = [column_info[0] for column_info in self.cursor.description]
    types_row = [self.types_dict.get(column_info[1], "UNKNOWN") for column_info
in self.cursor.description]
    gotten_data = self.cursor.fetchall()
    final_data_rows = [tuple(types_row)] + gotten_data
    return columns_names, final_data_rows

```

Тестування

Задля підтвердження коректності виконання програми, необхідно провести тестування кожного з пунктів меню користувача.

Проведемо тестування 1-го пункту меню. Занесемо необхідні параметри задля отримання конкретних даних таблиць.

Внесення тестових даних 1-го пункту користувацького меню зображено на рисунку нижче:

```
----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option: 1

FROM 'table_name': Users
SELECT 'columns_names' or 'ALL': user_id, data_registration, email, last_name
WHERE 'conditions' or 'NONE': user_id < 9999
ORDER BY 'columns_names' or 'NONE': user_id
LIMIT 'number_rows' or 'ALL': ALL
```

Рисунок 7 – Внесення тестових даних 1-го пункту користувацького меню

Результати роботи програми після внесення даних до 2-го пункту користувацького меню зображено на рисунку нижче:

```
### --- TABLE: Users
```

user_id	data_registration	email	last_name
INT	TS WITH TZ	CHAR VAR	CHAR VAR
384	2022-11-26 15:45:00+02:00	aer@gmail.com	Ваніна
1243	2020-11-22 10:45:00+02:00	awe@gmail.com	Шевченко
1244	2022-10-21 14:30:00+03:00	hrtow@gmail.com	Веранченко
1256	2024-12-26 14:45:00+02:00	jio@gmail.com	Кропивницький
2375	2023-10-26 15:45:00+03:00	loma@gmail.com	Петрушенко
6666	2022-11-26 15:45:00+02:00	ppppp	ппппп
8888	2022-11-26 15:45:00+02:00	lolo@gmail.com	Тропа

Рисунок 8 – Результати роботи програми після внесення даних до 1-го пункту користувацького меню

Порівняння результатів роботи програми та загального вигляду таблиці users у PostgreSQL зображено на рисунку нижче:

	user_id [PK] integer	first_name character varying (50)	last_name character varying (50)	data_registration timestamp with time zone	email character varying (50)
1	384	'Антон'	Ваніна	2022-11-26 15:45:00+02	aer@gmail.com
2	1243	Рома	Шевченко	2020-11-22 10:45:00+02	awe@gmail.com
3	1244	Віка	Веранченко	2022-10-21 14:30:00+03	hrtow@gmail.com
4	1256	Саша	Кропивницький	2024-12-26 14:45:00+02	jio@gmail.com
5	2375	Антон	Петрушенко	2023-10-26 15:45:00+03	loma@gmail.com
6	6666	Флла	ПППП	2022-11-26 15:45:00+02	ppppp
7	8888	Трini	Тропа	2022-11-26 15:45:00+02	lolo@gmail.com
8	9999	oi	loli	2022-11-26 15:45:00+02	fff@gmail.com
9	10019	Alice	Doe	2025-10-02 17:41:37.37869+03	alice.doe30@generated.com
10	10020	John	Smith	2025-07-23 19:32:05.757628+...	john.smith224@generated.com
11	10023	John	Williams	2025-07-10 20:55:15.108515+...	john.williams260@generated.c...
12	10032	Peter	Smith	2025-07-21 11:05:42.543622+...	peter.smith231@generated.com
13	10034	Peter	Jones	2025-08-25 19:55:58.541401+...	peter.jones134@generated.com
14	10040	Alice	Jones	2025-08-16 15:40:40.196943+...	alice.jones159@generated.com
15	10043	Alice	Davis	2025-09-28 01:07:13.966448+...	alice.davis43@generated.com
16	10046	Chris	Jones	2025-08-15 06:39:58.470931+...	chris.jones163@generated.com

Рисунок 9 – Порівняння результатів роботи програми та загального вигляду таблиці Users у PostgreSQL

Внесення тестових даних 1-го пункту користувацького меню зображено на рисунку нижче:

```

----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option: 1

FROM 'table_name': Lol
SELECT 'columns_names' or 'ALL': asdfef f
WHERE 'conditions' or 'NONE': aesv ad
ORDER BY 'columns_names' or 'NONE': av
LIMIT 'number_rows' or 'ALL': av s

```

Рисунок 10 – Внесення тестових даних 1-го пункту користувацького меню

Результат роботи програми внаслідок введення некоректних даних зображено на рисунку нижче:

```

# - Syntax ERROR . . .

```

Рисунок 11 – Результат роботи програми внаслідок введення некоректних даних

Проведемо тестування 2-го пункту меню за подібним принципом

Внесення тестових даних 2-го пункту користувацького меню зображено на рисунку нижче:

```
----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option: 2

INSERT INTO 'table_name': Users
VALUES 'value_1,value_2,...;': 1234,'ores','luli','2022-11-26 15:45:00+02','erggv@gmail.com'

# -> Дані успішно додано!
Натисніть Enter для продовження...
```

Рисунок 12 – Внесення тестових даних 2-го пункту користувацького меню

Результат роботи програми на прикладі отриманих даних PostgreSQL зображено на рисунку нижче:

2	1234	ores	luli	2022-11-26 15:45:00+02	erggv@gmail.com
---	------	------	------	------------------------	-----------------

Рисунок 13 – Результат роботи програми на прикладі отриманих даних PostgreSQL

Внесення некоректних даних до 2-го пункту користувацького меню зображено на рисунку нижче:

```
----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option: 2

INSERT INTO 'table_name': Users
VALUES 'value_1,value_2,...;': 1234

# -> Помилка під час вставки: the query has 5 placeholders but 1 parameters were passed
Натисніть Enter для продовження...
```

Рисунок 14 – Внесення некоректних даних до 2-го пункту користувацького меню

Проведемо тестування 3-го пункту меню.

Внесення некоректних даних до 3-го пункту користувацького меню зображено на рисунку нижче:

```
----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option: 3
FROM 'table_name': Users
WHERE 'conditions': user_id = 384

# -> Сталася помилка бази даних: update або delete в таблиці "Users" порушує обмеження зовнішнього ключа "user_id_fk" таблиці "Review"
DETAIL: На ключ (user_id)=(384) все ще є посилання в таблиці "Review".
Натисніть Enter для продовження...
```

Рисунок 15 – Внесення некоректних даних до 3-го пункту користувацького меню

Аби видалити дані з батьківської таблиці, необхідно аби на ці дані не посилалися жодні дочірні таблиці. Таким чином зберігається цілісність даних.

Проведемо тестування 4 та 5 пунктів

Внесення тестових даних до 4-го пункту користувацького меню зображено на рисунку нижче:

```
----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option: 4
UPDATE 'table_name': Users
SET 'column_1, column_2, column_3, ...': user_id
VALUES 'value_1, value_2, value_3, ...': 123
WHERE 'conditions': user_id = 1234

# -> Успішно оновлено 1 рядків.
Натисніть Enter для продовження...
```

Рисунок 16 – Внесення тестових даних до 4-го пункту користувацького меню

Внесення тестових даних до 5-го пункту користувацького меню зображено на рисунку нижче:

```

----- MENU -----
1. Get table data
2. Add new data
3. Delete data
4. Update data
5. Generate random data
Choose your option: 5
Enter number of new data: 15

# -> Починаю генерацію даних...
# -> Заповнюю таблицю "Users"...
# -> Заповнюю таблицю "Property owner"...
# -> Заповнюю таблицю "Realty"...
# -> Заповнюю таблицю "Review"...
# -> Заповнюю таблицю "Booking"...

# -> Успішно згенеровано та додано дані до всіх таблиць!
Натисніть Enter для продовження...

```

Рисунок 17 – Внесення тестових даних до 5-го пункту користувацького меню

Результат роботи програми у PostgreSQL після внесення даних до 4-го та 5-го пунктів користувацького меню зображено на рисунку нижче:

	user_id [PK] integer	first_name character varying (50)	last_name character varying (50)	data_registration timestamp with time zone	email character varying (50)
1	384	Антон	Ваніна	2022-11-26 15:45:00+02	aer@gmail.com
2	1234	ores	luli	2022-11-26 15:45:00+02	erggv@gmail.com
3	1243	Рома	Шевченко	2020-11-22 10:45:00+02	awe@gmail.com
4	1244	Віка	Веранченко	2022-10-21 14:30:00+03	hrtow@gmail.com
5	1256	Саша	Кропивницький	2024-12-26 14:45:00+02	jio@gmail.com
6	2375	Антон	Петрушенко	2023-10-26 15:45:00+03	loma@gmail.com
7	6666	Флла	ппппп	2022-11-26 15:45:00+02	ppppp
8	8888	Тріні	Тропа	2022-11-26 15:45:00+02	lolo@gmail.com
9	9999	oi	loli	2022-11-26 15:45:00+02	fff@gmail.com
10	10019	Alice	Doe	2025-10-02 17:41:37.37869+03	alice.doe30@generated.com
11	10020	John	Smith	2025-07-23 19:32:05.757628+...	john.smith224@generated.com
12	10023	John	Williams	2025-07-10 20:55:15.108515+...	john.williams260@generated.c...
13	10032	Peter	Smith	2025-07-21 11:05:42.543622+...	peter.smith231@generated.com
14	10034	Peter	Jones	2025-08-25 19:55:58.541401+...	peter.jones134@generated.com
15	10040	Alice	Jones	2025-08-16 15:40:40.196943+...	alice.jones159@generated.com
16	10043	Alice	Davis	2025-09-28 01:07:13.966448+...	alice.davis43@generated.com

Рисунок 18 – Результат роботи програми у PostgreSQL після внесення даних до 4-го та 5-го пунктів користувацького меню

Посилання на GitHub:

<https://github.com/liashenkodaniil/->

Контакти в телеграм:

Username: @daniczek

Nickname: Danchik

Електронна пошта:

kyvalich@gmail.com